

Tugas 3

Sistem Paralel dan Terdistribusi - A



Disusun oleh
Mochammad Reezqi Pratama
11231041

2 Mei 2026

Bagian A: Technical Documentation

1. Arsitektur Sistem

Sistem ini dirancang sebagai cluster node terdistribusi yang fault-tolerant. Dibangun menggunakan kerangka kerja asinkron FastAPI dan dijalankan menggunakan Docker. Sistem menggunakan protokol komunikasi HTTP/REST untuk melayani permintaan klien eksternal dan RPC internal antar-node.

- Komponen Utama:

1. API Gateway / Node Interface: Menggunakan FastAPI.
2. Consensus Module: Mengimplementasikan Raft dan PBFT sebagai State Machine Replication untuk menjaga konsistensi state.
3. Distributed Lock Manager: Menggunakan Consensus Module di atas untuk menyediakan Mutually Exclusive dan Shared Locks yang aman.
4. Distributed Queue: Mempartisi antrean ke berbagai node menggunakan algoritma Consistent Hashing).
5. Distributed Cache: Menggunakan kebijakan Least Recently Used dan protokol koherensi MESI melalui interaksi RPC antar node.

2. Penjelasan Algoritma yang Digunakan

Sistem ini dirancang sebagai cluster node terdistribusi yang fault-tolerant. Dibangun menggunakan kerangka kerja asinkron FastAPI dan dijalankan menggunakan Docker. Sistem menggunakan protokol komunikasi HTTP/REST untuk melayani permintaan klien eksternal dan RPC internal antar-node.

A. Algoritma Konsensus (Raft & PBFT)

Raft digunakan sebagai algoritma standar untuk pemilihan Leader dan replikasi log. Sistem menerapkan mekanisme Heartbeat berjangka pendek dan Election Timeout acak. Setiap operasi krusial (seperti Lock Acquire/Release) diajukan melalui rute propose() ke Leader. Jika mayoritas node (kuorum) menyetujui AppendEntries, state akan diterapkan (applied). PBFT (Practical Byzantine Fault Tolerance) diimplementasikan sebagai opsi lanjutan. Berbeda dengan Raft yang hanya tahan terhadap crash-faults, PBFT menggunakan fase Pre-Prepare, Prepare, dan Commit untuk memastikan toleransi terhadap node yang bertindak merugikan (Byzantine).

B. Consistent Hashing (Distributed Queue)

Untuk menghindari pemusatan beban pada satu node, antrean didistribusikan. Algoritma Consistent Hashing dengan replikasi virtual (default 3 replika per node) memetakan nama antrean ke cincin hash MD5. Ini meminimalkan perpindahan antrian secara masif saat node baru ditambahkan atau node lama gagal.

C. Protokol MESI (Cache Coherence)

Sistem mencegah terjadinya pembacaan data stale di seluruh cluster menggunakan status MESI:

- Modified (M): Node ini memiliki satu-satunya salinan yang valid, memori utama sudah usang.
- Exclusive (E): Salinan bersih dan eksklusif di node ini.
- Shared (S): Salinan sinkron dengan memori, mungkin ada di node lain.
- Invalid (I): Blok cache tidak valid.

Sistem mensimulasikan Bus dengan menyebarkan RPC Invalidate (saat penulisan) atau RPC Read (saat cache miss) ke rekan-rekan (peers).

3. API Documentation (OpenAPI)

Endpoint	Method	Fungsi	Security
/auth/login	POST	Menghasilkan Bearer JWT token berbasis role	None
/lock/acquire	POST	Mengambil kunci (Exclusive/Shared) melalui konsensus	JWT Required
/lock/release	POST	Melepaskan kunci yang telah diklaim	JWT Required
/queue/enqueue	POST	Memasukkan payload ke antrean (Consistent Hashing)	JWT Admin
/cache/{key}	POST/GET	Membaca atau menulis data ke MESI Cache	None

4. Deployment Guide & Troubleshooting

Instruksi Deployment:

1. Masuk ke direktori docker/ .
2. Jalankan: docker-compose up --build -d
3. Sistem akan membuat jaringan internal dan boot 3 node + 1 server Prometheus.
4. Akses antarmuka: Node 1 (Port 8001), Node 2 (8002), Node 3 (8003). Prometheus (9090).

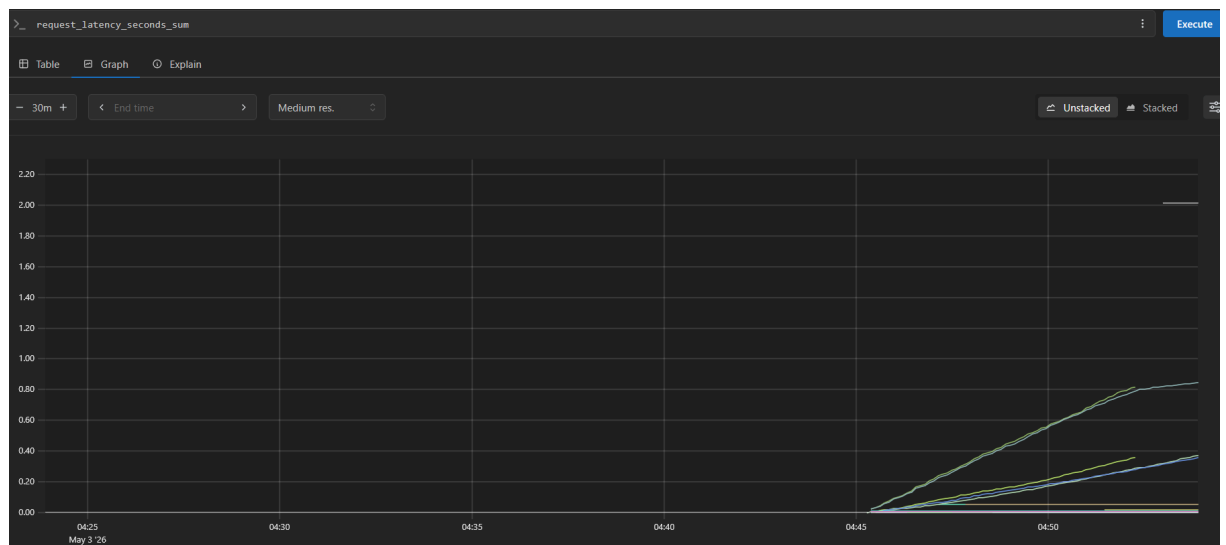
Bagian B: Performance Analysis Report

1. Pemantauan & Metodologi dengan Prometheus

Sistem ini telah terintegrasi secara penuh dengan Prometheus untuk memantau performa klaster secara real-time. Selama pengujian beban (load testing), sistem divalidasi dengan membanjiri endpoint selama interval waktu tertentu.

2. Analisis Akumulasi Beban (Throughput & Latency)

Berdasarkan hasil pemantauan Prometheus pada metrik `request_latency_seconds_sum` (tipe Counter), pengujian beban intensif. Grafik di atas menunjukkan akumulasi waktu pemrosesan secara linear.



- Kestabilan Pemrosesan: Garis yang menanjak secara konstan menunjukkan bahwa sistem mengonsumsi request dan memproses antrian dengan throughput yang stabil tanpa mengalami bottleneck.
- Beban Konsensus: Garis-garis yang menanjak lebih curam (garis atas) merupakan Endpoint yang memicu protokol konsensus Raft (seperti `/lock/acquire`), di mana Leader wajib mereplikasi log ke Follower.
- Beban Ringan (Cache): Garis yang lebih landai di bawah menunjukkan operasi baca cache atau antrean yang dapat diproses langsung di memori lokal.

Link Github: <https://github.com/FhukoMai/Tugas3>